In [8]:
```python
# a. Import necessary packages
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense ,Flatten
```

In [9]:
```python
# b. Load training and testing data
train = pd.read_csv('mnist_train.csv')
test = pd.read_csv('mnist_test.csv')

# Separate images and labels
X_train = train.iloc[:, 1:].values
y_train = train.iloc[:, 0].values

X_test = test.iloc[:, 1:].values
y_test = test.iloc[:, 0].values

# Normalize pixel values
X_train = X_train / 255.0
X_test = X_test / 255.0
```

In [10]:
```python
# c. Define the network architecture
model = Sequential([
    Dense(128, activation='relu', input_shape=(784,)),
    Flatten(),
    Dense(64, activation='relu'),
    Dense(10, activation='softmax')
])

# Compile the model
model.compile(
    optimizer='sgd',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

In [11]:
```python
# d. Train the model for 11 epochs
history = model.fit(
    X_train, y_train,
    epochs=11,
    batch_size=32,
    validation_split=0.1
)
# e. Evaluate the model
loss, accuracy = model.evaluate(X_test, y_test)

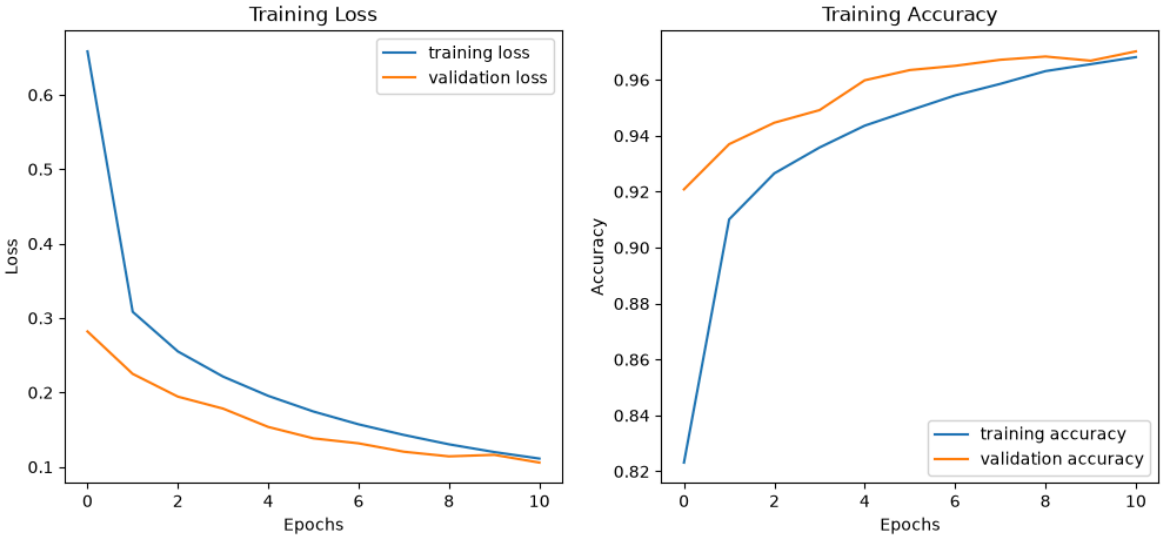
print("Test Loss:", loss)
print("Test Accuracy:", accuracy * 100, "%")
```

```
Epoch 1/11
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 8s 3ms/step - accuracy: 0.8231 - loss: 0.6584 - va
l_accuracy: 0.9208 - val_loss: 0.2819
Epoch 2/11
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.9101 - loss: 0.3084 - va
l_accuracy: 0.9370 - val_loss: 0.2250
Epoch 3/11
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9266 - loss: 0.2551 - va
l_accuracy: 0.9447 - val_loss: 0.1944
Epoch 4/11
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.9358 - loss: 0.2213 - va
l_accuracy: 0.9492 - val_loss: 0.1783
Epoch 5/11
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9436 - loss: 0.1954 - va
l_accuracy: 0.9598 - val_loss: 0.1537
Epoch 6/11
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9491 - loss: 0.1744 - va
l_accuracy: 0.9635 - val_loss: 0.1383
Epoch 7/11
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.9544 - loss: 0.1572 - va
l_accuracy: 0.9650 - val_loss: 0.1316
Epoch 8/11
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9586 - loss: 0.1429 - va
l_accuracy: 0.9672 - val_loss: 0.1204
Epoch 9/11
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.9631 - loss: 0.1303 - va
l_accuracy: 0.9683 - val_loss: 0.1141
Epoch 10/11
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.9656 - loss: 0.1199 - va
l_accuracy: 0.9668 - val_loss: 0.1162
Epoch 11/11
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9681 - loss: 0.1112 - va
l_accuracy: 0.9702 - val_loss: 0.1058
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9642 - loss: 0.1189
Test Loss: 0.11891666054725647
Test Accuracy: 96.42000198364258 %
```

In [18]:

```python
# f. Plot training loss and accuracy
plt.figure(figsize=(12, 5))

# Loss graph
plt.subplot(1, 2, 1)
plt.plot(history.history['loss'],label='training loss')
plt.plot(history.history['val_loss'],label='validation loss')
plt.title('Training Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
# Accuracy graph
plt.subplot(1, 2, 2)
plt.plot(history.history['accuracy'],label='training accuracy')
plt.plot(history.history['val_accuracy'],label='validation accuracy')
plt.title('Training Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```


Training Loss
training loss
validation loss
Loss
Epochs
Training Accuracy
training accuracy
validation accuracy
Accuracy
Epochs

In [ ]: